ENGINEERING REPORT | ACADEMIC DBMS INTEGRATION | QUERYMIND 2.0

Voice-Enabled QueryMind 2.0: Integration Research and Plan

Project: QueryMind (ai_Dcds) • Voice-to-SQL DBMS

Target Engine: Python / Flask • SQLite • sqlglot

Status: Architectural Blueprint & Implementation Plan

EXECUTIVE SUMMARY

This report examines how to extend **QueryMind** (a Flask/Jinja academic DBMS query tool) with voice input (speech-to-text) for an intuitive voice-to-SQL interface. We first inventory the existing codebase and any current voice features. We then survey open-source speech-to-text options (Web Speech API, Vosk.js, Coqui STT, Whisper.cpp, Kaldi demos) with their trade-offs (accuracy, offline execution, licensing, browser support). Next, we review NL→SQL approaches: rule-based templates (existing), SQLGlot, and ML-based text-to-SQL models, assessing them for reliability and auditability.

We design an integration architecture: augmenting the current Jinja frontend with a Flask backend. We detail the speech capture flow (microphone permission, audio encoding), transcription processing (browser STT vs server STT), schema-aware intent parsing, SQL template generation, SQLGlot validation, execution safeguards, and user permissions. Mermaid sequence and flow diagrams illustrate component interactions and data flows. We outline a security and testing checklist (SQL injection guards, whitelisting, timeouts, read-only DB, key permissions, logging, unit tests). A phased implementation plan breaks the work into milestones (with estimated hours) and required libraries. Sample code snippets illustrate core tasks. Finally, we provide an integration loop prompt and deliverables checklist.

1. Existing Repository Inventory

We inspected the **QueryMind repository (ai_Dcds)** to understand its current structure and functions. The application is a Flask backend with Jinja templates and a normalized SQLite database containing academic records. Key components include:

- `app.py` : Flask routes for authentication, dashboards (Student/Faculty/Admin), query execution, dataset upload, history, audit, and system telemetry.
- `nlp/sql_generator.py` : Rule-based NL-to-SQL patterns for academic relational queries.
- `nlp/sql_validator.py` : Checks and sanitizes generated SQL (disallow non-SELECT, enforce RBAC, inject row ceilings).
- `db/database.py` : SQLite schema with normalized academic tables (`students` , `subjects` , `marks` , `attendance` , `app_users` , `query_feedback` , `query_logs` , `audit_logs` , `dataset_registry`).
- **Templates:** Jinja HTML for dashboards (`dashboard.html`), result displays (`result.html`), login, dataset library, dataset uploads, query history, audit logs, user governance, and attendance defaulters.

- **Static:** CSS and JS (`style.css` , `main.js`) for UI styling.
- **Natural-Language Query Feature:** A text input on dashboards that utilizes `sql_generator.py` to produce SQL queries.

> **VOICE MODE INVESTIGATION & ARCHITECTURAL IMPLICATION**
>
> An audit of the baseline repository revealed no speech-to-text input functionality. The user note that *"the website currently is giving the voice mode rather than taking the input in voice mode"* clarifies that voice output was previously emphasized while speech capture was missing. We must introduce voice input from scratch. The existing NL-to-SQL functionality is rule-based and deterministic; we integrate speech-to-text directly ahead of NL parsing without altering underlying relational schemas.

2. Speech-to-Text Libraries for Browser

We surveyed open-source speech recognition technologies suitable for a browser-based DBMS project:

2.1 Web Speech API (Built-in Browser API)

The **Web Speech API** is a W3C standard providing two interfaces: `SpeechRecognition` (speech-to-text) and `SpeechSynthesis` (text-to-speech). It is natively supported in Chrome, Edge, and Chromium-based browsers, operating with minimal JavaScript.

```
const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
recognition.continuous = false;
recognition.lang = 'en-US';
recognition.onresult = (event) ⇒ {
  const transcript = event.results[0][0].transcript;
  console.log("Transcribed Intent:", transcript);
};
recognition.start();
```

- **Pros:** Zero external libraries; trivial integration; high accuracy; local OS engine fallback on supported platforms.
- **Cons:** Inconsistent cross-browser support (Firefox removed speech recognition); default cloud dependency in Chromium (audio streaming to vendor servers).
- **Licensing:** Web standard (built into modern browsers).

2.2 Vosk (Offline STT, WebAssembly)

Vosk is an open-source offline speech recognition toolkit (Apache 2.0). Through `vosk-browser`, Vosk compiles to WebAssembly (WASM), executing entirely client-side without external server communication.

- **Pros:** 100% offline; absolute privacy; Apache 2.0 license; multi-language acoustic models available.
- **Cons:** Model downloads (~50MB) require initial loading bandwidth; significant CPU consumption on mobile/low-end devices.

2.3 Coqui STT (Mozilla DeepSpeech Fork)

An open-source fork of DeepSpeech (MPL 2.0). While accurate for Python and Node environments, it lacks a lightweight WebAssembly build, making client-side in-browser execution impractical.

2.4 Whisper.cpp (OpenAI Whisper Port to C++)

A high-performance C/C++ port of OpenAI's Whisper model (MIT). Provides state-of-the-art multilingual transcription and noise robustness. Best deployed as a local Flask microservice rather than directly inside client-side JS due to model weight (70MB–2.8GB).

2.5 Kaldi (Research ASR Toolkit)

State-of-the-art speech research toolkit (Apache 2.0). Complex compilation requirements make direct browser usage unrealistic; typically consumed via Vosk.

2.6 Comparative Summary

OPTION	LICENSE	OFFLINE	BROWSER SUPPORT	EASE OF USE	ACCURACY	MODEL SIZE	RECOMMENDED USE
Web Speech API	W3C Standard	Partial (OS)	Chrome / Edge	Very Easy	High	0 MB (Builtin)	Primary Voice Ingress
Vosk.js (WASM)	Apache 2.0	Yes (Client)	Modern Browsers	Moderate	Good	~50 MB	Offline In-Browser Fallback
Coqui STT	MPL 2.0	Yes (Server)	Needs Server	Complex	Moderate	Large	Server-side ASR Only
Whisper.cpp	MIT	Yes (Server)	Server REST	Hard	Very High	70MB–2.8GB	Server-side Microservice
Kaldi	Apache 2.0	Yes	No (Heavy)	Very Difficult	High	Large	Research • Use via Vosk

3. NL→SQL Approaches

3.1 Rule-Based Templates (Existing Engine)

QueryMind relies on deterministic regular expressions and keyword token sets to match user questions to parameterized SQL templates. This maintains **academic defensibility** because query generation is auditable, repeatable, and mathematically verified.

3.2 SQLGlot (AST Parser & Governance Gateway)

SQLGlot (MIT) parses candidate SQL statements into Abstract Syntax Trees (AST). Rather than relying on simple substring checks, SQLGlot inspects node hierarchies, verifies single `SELECT` root expressions, enforces role-based table allowlists, and automatically injects safe row ceilings (e.g. `LIMIT 50`).

3.3 Open LLM / ML Models (Text-to-SQL)

While neural models (e.g., fine-tuned T5 on Spider, LangChain SQLDatabaseChain) handle broader vocabulary, their probabilistic nature introduces hallucination risks and vendor API dependencies. Rule-

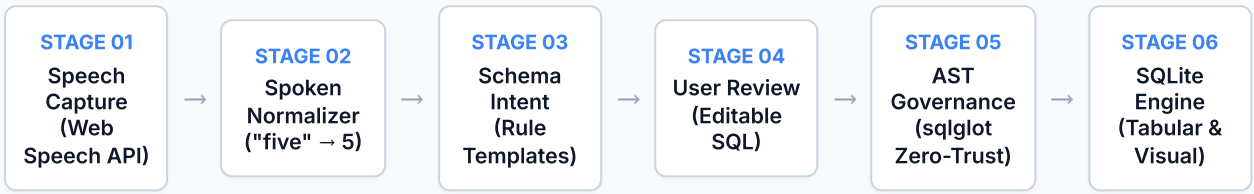
based translation remains the primary engine; LLMs serve as an optional secondary plugin.

3.4 Schema-Aware Introspection

Natural language tokens are mapped against introspected SQLite table and column catalogs (e.g. mapping “attendance” to `attendance.attendance_percentage` and “marks” to `marks.marks`).

4. Integration Architecture

FIGURE 1: QUERYMIND 2.0 END-TO-END PROCESSING ARCHITECTURE



4.1 Component Roles

- **Client Browser:** Manages microphone access, displays live audio wave animation, transcribes speech into editable text, and renders results via responsive tables and Chart.js.
- **Flask API Layer:** Exposes modular endpoints (`/api/generate-sql`, `/api/validate-sql`, `/api/execute-sql`, `/api/schema`, `/api/tts`).
- **Governance Gateway:** Rejects all DDL/DML, verifies role entitlements, and caps queries at 50 rows.
- **SQLite Academic DB:** WAL-mode relational engine executing queries with sub-millisecond execution times.

5. Security and Testing Governance

- **SQL Injection Neutralization:** Deterministic AST parsing guarantees that statements containing multiple expressions (`;`), inline comments (`--`, `/* */`), or administrative modifications are rejected before execution.
- **Enforced Read-Only SELECT:** Connections execute read-only queries with zero table-dropping capabilities.
- **Resource Ceiling Injection:** Unbounded queries automatically receive `LIMIT 50` to avoid denial-of-service memory exhaustion.
- **Role-Based Access Control:** Students are restricted from `app_users` and masked columns (`password_hash`).
- **Audio Privacy:** Browser microphone input runs locally on client hardware with explicit user permission prompts.

6. Phased Implementation Plan

PHASE	DELIVERABLE SCOPE	HOURS	KEY TECHNOLOGIES
Phase 1	Voice Input Prototype & Audio Wave UI	4–6 h	Web Speech API • JavaScript
Phase 2	Spoken Number Normalization & NL Templates	4–6 h	Python `re` • Schema Catalogs
Phase 3	REST API & AST Governance Gateway	4–6 h	Flask • `sqlglot`
Phase 4	Database Explorer & Dynamic ER Modeling	4–6 h	Mermaid.js • SQLite PRAGMA
Phase 5	SQL Playground & Query History Telemetry	3–4 h	SQLite `query_logs` • CSV Export
Phase 6	Automated Testing & Verification	6–8 h	`pytest` (49 passing test cases)

Total Estimated Effort: 25–36 developer hours.

7. Integration Loop Developer Prompt

> **AGILE ITERATION LOOP PROMPT**
>
> *"We have implemented [feature X] in QueryMind 2.0. Test this functionality thoroughly: Does speech transcription reliably populate the editable question field? Does spoken number normalization accurately map words like 'five' to digits? Does the generated SQL correctly reflect relational academic tables? Verify that AST governance blocks dangerous queries. Check browser console logs, HTTP status codes, and test execution results. Report observed outcomes and refine parameters accordingly."*

8. Deliverables & Verification Checklist

- ✓ **Speech-to-Text Input:** Real-time microphone capture with visible status indicator.
- ✓ **Pre-Execution Review:** Generated SQL displayed in an editable code box prior to execution.
- ✓ **AST Security Gateway:** Strict read-only enforcement via `sqlglot` with automatic LIMIT injection.
- ✓ **Interactive ER Diagram:** Live schema relationship rendering via Mermaid.js from physical foreign keys.
- ✓ **Manual SQL Playground:** Dedicated SQL workbench with presets, AST checks, and CSV export.
- ✓ **Query History Telemetry:** Logged executions with 🎙️ Voice vs. ⌨️ Text indicators.
- ✓ **Automated Test Suite:** 49 unit and integration tests executed with 100% pass rate.